

AWS Enterprise Transformation Playbook

Generate Pdfs

Generated: February 12, 2026

Generate PDF Documents from Playbook Artifacts

This guide explains how to convert the markdown artifacts in this playbook to professional PDF documents.

Quick Start

1. Install Dependencies

```
# Navigate to playbook directory
cd enterprise-architecture/transformation/aws-enterprise-playbook
# Install required Python packages
pip install -r requirements.txt
```

2. Convert Individual File

```
# Convert the master README to PDF
python automation/scripts/markdown_to_pdf.py --input README.md
# Convert business case template
python automation/scripts/markdown_to_pdf.py \
--input business-case/business-case-template.md \
--output business-case.pdf
```

3. Convert All Artifacts

```
# Convert all markdown files to PDF
python automation/scripts/markdown_to_pdf.py \
--directory . \
--output-dir ./pdf-outputs
```

Conversion Examples

Convert Specific Playbooks

```
# Convert all phase playbooks
python automation/scripts/markdown_to_pdf.py \
--directory playbooks/ \
--output-dir pdf-playbooks/
```

Convert Business Case Artifacts

```
# Convert all business case documents
python automation/scripts/markdown_to_pdf.py \
--directory business-case/ \
--output-dir pdf-business-case/
```

Convert Governance Documents

```
# Convert all governance artifacts
python automation/scripts/markdown_to_pdf.py \
--directory governance/ \
--output-dir pdf-governance/
```

Use A4 Page Size

```
# Convert with A4 page size (default is letter)
python automation/scripts/markdown_to_pdf.py \
--input README.md \
--pagesize A4
```

PDF Features

The generated PDFs include:

Professional Formatting

- ■ Custom headers and footers
- ■ Page numbers
- ■ Generation timestamp
- ■ Professional typography

- ■ Color-coded headings

Content Support

- ■ Headings (H1-H4)
- ■ Paragraphs with justification
- ■ Bullet lists and numbered lists
- ■ Tables with styling
- ■ Code blocks with syntax highlighting
- ■ Block quotes
- ■ Horizontal rules

Document Structure

- ■ Automatic cover page
- ■ Document title from filename
- ■ Table of contents (for longer documents)
- ■ Consistent spacing and margins

Use Cases

For Executives

Generate PDFs for board presentations:

```
# Business case for board approval
python automation/scripts/markdown_to_pdf.py \
--input business-case/business-case-template.md \
--output board-presentation.pdf
# Executive summary
python automation/scripts/markdown_to_pdf.py \
--input business-case/executive-summary-template.md \
--output executive-summary.pdf
```

For Compliance Audits

Generate compliance documentation:

```
# All compliance checklists
python automation/scripts/markdown_to_pdf.py \
--directory governance/compliance/ \
--output-dir compliance-pdfs/
```

For Training Materials

Convert training curricula to PDF:

```
# Training materials
python automation/scripts/markdown_to_pdf.py \
--directory change-management/training-curriculum/ \
--output-dir training-pdfs/
```

For Stakeholder Distribution

Create full playbook PDF package:

```
# Generate all PDFs for distribution
python automation/scripts/markdown_to_pdf.py \
--directory . \
--output-dir playbook-pdfs/
```

Batch Conversion Script

Create a shell script to convert all key artifacts:

```
#!/bin/bash
# convert-all-pdfs.sh
echo "■ Converting all playbook artifacts to PDF..."
# Create output directory
mkdir -p pdf-outputs
# Master documentation
python automation/scripts/markdown_to_pdf.py \
--input README.md \
--output pdf-outputs/00-Master-Playbook.pdf
python automation/scripts/markdown_to_pdf.py \
--input ARTIFACTS-CATALOG.md \
--output pdf-outputs/00-Artifacts-Catalog.pdf
# Business case
```

```
python automation/scripts/markdown_to_pdf.py \
--directory business-case/ \
--output-dir pdf-outputs/01-business-case/
# Governance
python automation/scripts/markdown_to_pdf.py \
--directory governance/ \
--output-dir pdf-outputs/02-governance/
# Change management
python automation/scripts/markdown_to_pdf.py \
--directory change-management/ \
--output-dir pdf-outputs/03-change-management/
# Assessment & Planning
python automation/scripts/markdown_to_pdf.py \
--directory assessment/ \
--output-dir pdf-outputs/04-assessment/
python automation/scripts/markdown_to_pdf.py \
--directory planning/ \
--output-dir pdf-outputs/05-planning/
# Playbooks
python automation/scripts/markdown_to_pdf.py \
--directory playbooks/ \
--output-dir pdf-outputs/06-playbooks/
echo "■ PDF conversion complete! Check pdf-outputs/ directory"
```

Make it executable and run:

```
chmod +x convert-all-pdfs.sh
./convert-all-pdfs.sh
```

Troubleshooting

Missing Dependencies

```
# If conversion fails, install dependencies manually
pip install reportlab markdown beautifulsoup4 lxml
```

Font Issues

```
# If fonts don't render correctly, install system fonts
# Ubuntu/Debian
sudo apt-get install fonts-liberation
# macOS
# Fonts are included by default
# Windows
# Fonts are included by default
```

Memory Issues for Large Files

```
# For very large markdown files, split them first
# Or increase Python memory limit
export PYTHONHASHSEED=0
python -X dev automation/scripts/markdown_to_pdf.py --input large-file.md
```

Advanced Usage

Custom Styling

Modify `markdown_to_pdf.py` to customize: - Colors (search for `HexColor` values) - Fonts (modify `fontName` parameters) - Spacing (adjust `spaceAfter`, `spaceBefore`) - Page size (change `pagesize` parameter)

Add Watermarks

Add watermark to PDFs:

```
# Add to add_header_footer method
canvas.setFont('Helvetica-Oblique', 60)
canvas.setFillColorRGB(0.9, 0.9, 0.9)
canvas.saveState()
canvas.translate(doc.width/2, doc.height/2)
canvas.rotate(45)
canvas.drawCentredString(0, 0, "DRAFT")
canvas.restoreState()
```

Merge Multiple PDFs

```
# Install PyPDF2
pip install PyPDF2
# Merge script (create merge-pdfs.py)
from PyPDF2 import PdfMerger
merger = PdfMerger()
for pdf in ['file1.pdf', 'file2.pdf', 'file3.pdf']:
    merger.append(pdf)
merger.write("merged-playbook.pdf")
merger.close()
```

Distribution

Email Distribution

```
# Generate PDFs and email to stakeholders
python automation/scripts/markdown_to_pdf.py --input README.md
# Then attach to email
```

SharePoint/Confluence Upload

```
# Generate PDFs
python automation/scripts/markdown_to_pdf.py --directory . --output-dir pdfs/
# Upload to SharePoint/Confluence via their APIs
# (requires additional scripting)
```

Version Control

```
# Add PDFs to git (optional, generally not recommended for version control)
# Better: Generate on-demand from markdown
# Or create releases
git tag -a v1.0 -m "Playbook Version 1.0"
git push origin v1.0
```

Best Practices

1. Keep Markdown Source: Always maintain markdown as source of truth

2. Generate PDFs On-Demand: Don't commit PDFs to version control
3. Date PDFs: Use timestamps in filenames for tracking
4. Test Before Distribution: Review PDFs before sending to stakeholders
5. Use Consistent Naming: Follow naming convention for output files

Support

For issues or questions: 1. Check requirements.txt is fully installed 2. Verify markdown syntax is valid 3. Review error messages for specific issues 4. Consult README.md for playbook usage

Last Updated: 2026-02-12 Script Version: 1.0 Supported Formats: Markdown (.md) → PDF (.pdf)